Adaptive Step Control for RK Methods

Andrea Tribotti, Leonardo Cittadini, Werther Solazzi
Department of Mathematics: Numerical Methods Exam
University of Bologna

March 2026

Introduzione

This report presents an implementation and comparison of adaptive step size control strategies for explicit Runge-Kutta (ERK) methods applied to the numerical solution of initial value problems (IVPs). Adaptive step-size strategies are commonly adopted in numerical methods to balance computational efficiency with solution accuracy: smaller steps are used in regions where the solution varies rapidly, while larger steps are allowed when the dynamics are smooth[1].

The study is structured in two phases. In the first phase, we test three embedded Runge-Kutta tableaux—ERK4(3), Bogacki-Shampine BS3(2), and Dormand-Prince DP5(4)—on multiple test problems including the Brusselator system and two scalar ODEs. An interactive MATLAB application was developed to facilitate parameter exploration, allowing users to select the ODE, the embedded tableau, tolerance settings, and safety factor. In the second phase, we focus exclusively on the Brusselator to conduct a detailed comparison between adaptive and non-adaptive strategies, investigating three distinct implementations: a third-order ERK method (ERK3), a fourth-order ERK method (ERK4), and a fourth-order ERK method enhanced with Richardson extrapolation. The automatic step size control mechanism is based on the framework described by Hairer and Norsett[1], which compares numerical approximations of different orders to estimate the local truncation error and adjust the step size accordingly.

1 Theoretical Framework

Explicit Runge-Kutta Methods

An explicit Runge-Kutta method of order p for solving the initial value problem

$$y' = f(x, y), \quad y(x_0) = y_0$$

computes the numerical solution y_{n+1} at $x_{n+1} = x_n + h$ using the formula

$$y_{n+1} = y_n + h \sum_{i=1}^{s} b_i k_i$$

where the stages k_i are defined by

$$k_i = f\left(x_n + c_i h, y_n + h\sum_{j=1}^{i-1} a_{ij}k_j\right)$$

The coefficients a_{ij}, b_i, and c_i are chosen to satisfy order conditions that ensure the local truncation error behaves as $O(h^{p+1})$[1].

Richardson Extrapolation

Richardson extrapolation is a classical technique for improving the accuracy of numerical approximations by exploiting the known asymptotic behavior of the truncation error[1]. For a Runge-Kutta method of order p, if we compute two steps with step size h to obtain y_2, and one big step with step size $2h$ to obtain w, the errors satisfy

$$y(x_0 + 2h) - y_2 \approx 2Ch^{p+1}$$

$$y(x_0 + 2h) - w \approx C(2h)^{p+1}$$

where C depends on higher-order derivatives of the solution. By eliminating C, we obtain the extrapolated approximation

$$\hat{y}_2 = y_2 + \frac{y_2 - w}{2^p - 1}$$

which has accuracy $O(h^{p+2})$[1]. The difference $y_2 - w$ also provides an estimate of the error in y_2:

$$y(x_0 + 2h) - y_2 \approx \frac{y_2 - w}{2^p - 1}$$

Embedded Runge-Kutta Formulas

An alternative to Richardson extrapolation is to use embedded Runge-Kutta formulas, where two approximations y_1 and $\hat{y}_1$ of orders p and $\hat{p}$ (typically $\hat{p} = p - 1$ or $\hat{p} = p + 1$) share the same function evaluations[1]. The method is characterized by a Butcher tableau with two sets of weights:

$$\begin{array}{c|cccc}
0 & & & & \\
c_2 & a_{21} & & & \\
c_3 & a_{31} & a_{32} & & \\
\vdots & \vdots & \vdots & \ddots & \\
c_s & a_{s1} & a_{s2} & \cdots & a_{s,s-1} \\
\hline
 & b_1 & b_2 & \cdots & b_s \\
 & \hat{b}_1 & \hat{b}_2 & \cdots & \hat{b}_s
\end{array}$$

Table 1: Embedded Runge-Kutta tableau with orders $p/\hat{p}$

The difference $|y_1 - \hat{y}_1|$ provides an estimate of the local error and is used for step size control[1].

Automatic Step Size Control

The step size control algorithm adjusts h to keep the estimated local error within prescribed tolerances. Given absolute and relative tolerances Atol_i and Rtol_i for each component, we define the scaling factor

$$\mathrm{sc}_i = \mathrm{Atol}_i + \max(|y_{0,i}|, |y_{1,i}|) \cdot \mathrm{Rtol}_i$$

and compute the normalized error measure

$$\mathrm{err} = \sqrt{\frac{1}{n} \sum_{i=1}^{n} \left(\frac{y_{1,i} - \hat{y}_{1,i}}{\mathrm{sc}_i} \right)^2}$$

The optimal step size is estimated from the relation $\mathrm{err} \approx Ch^{q+1}$, where $q = \min(p, \hat{p})$, as

$$h_{\mathrm{opt}} = h \left(\frac{1}{\mathrm{err}} \right)^{1/(q+1)}$$

To ensure robustness, a safety factor fac (typically 0.8 or 0.9) is applied, and the step size change is limited:

$$h_{\mathrm{new}} = h \cdot \min\left(\mathrm{fac}_{\max}, \max\left(\mathrm{fac}_{\min}, \mathrm{fac} \cdot \mathrm{err}^{-1/(q+1)}\right)\right)$$

If $\mathrm{err} \leq 1$, the step is accepted and integration continues with y_1; otherwise, the step is rejected and recomputed with h_{new}[1].

Initial Step Size Selection

Rather than requiring the user to specify an initial step size, an automatic procedure estimates a reasonable starting value[1]. The algorithm is based on approximating the local error $Ch^{p+1}|y^{(p+1)}(x_0)|$ using estimates of the first and second derivatives:

1. Compute $f(x_0, y_0)$ and define $d_0 = \|y_0\|$ and $d_1 = \|f(x_0, y_0)\|$
2. Set $h_0 = 0.01 \cdot d_0/d_1$ (or $h_0 = 10^{-6}$ if d_0 or d_1 is very small)
3. Perform an explicit Euler step: $y_1 = y_0 + h_0 f(x_0, y_0)$, and evaluate $f(x_0 + h_0, y_1)$
4. Estimate the second derivative: $d_2 = \|f(x_0 + h_0, y_1) - f(x_0, y_0)\|/h_0$
5. Compute h_1 from $h_1^{p+1} = 0.01/\max(d_1, d_2)$
6. Select $h = \min(100h_0, h_1)$ as the initial step size

This procedure generally yields a reasonable initial step size and prevents extremely poor starting choices, which is crucial for the overall robustness of the adaptive ERK implementation[1].

2 Implementation

2.1 ERK(3): Third-Order Runge-Kutta Method

For the third-order implementation, we use a 3-stage explicit Runge-Kutta method. A commonly used third-order method is Kutta's third-order method:

0			
1/2	1/2		
1	-1	2	
	1/6	2/3	1/6

Table 2: Kutta's third-order method (ERK3)

To enable adaptive step size control, we embedded a second-order approximation by choosing alternative weights $\hat{b}_i$. For a 3-stage method with $c_2 = 1/2$, setting $\hat{b}_3 = 0$, $\hat{b}_2 = 1/(2c_2) = 1$, and $\hat{b}_1 = 0$ gives a second-order embedded formula[1].

2.2 ERK(4): Fourth-Order Runge-Kutta Method

For the fourth-order implementation, we use the classical RK4 method (also known as the 3/8 rule variant for embedded formulas):

0				
1/2	1/2			
1/2	0	1/2		
1	0	0	1	
	1/6	1/3	1/3	1/6

Table 3: Classical fourth-order Runge-Kutta method (RK4)

We constructed a third-order embedded formula using the FSAL (First Same As Last) property, where the last stage of one step serves as the first stage of the next step. The embedded weights are determined by solving the third-order conditions with the constraint $b_4 = 0$[1]:

$$\hat{b}_1 = 2b_1 - \frac{1}{6}, \quad \hat{b}_2 = \frac{1}{2c_2} - b_2, \quad \hat{b}_3 = \frac{1}{2c_3} - b_3, \quad \hat{b}_4 = 0, \quad \hat{b}_5 = \frac{1}{6}$$

2.3 ERK(4) with Richardson Extrapolation

In addition to the embedded formula approach, we implemented a fourth-order method with Richardson extrapolation. After computing two steps of size h to reach $x_n + 2h$, we also compute one big step of size $2h$ from x_n. For a fourth-order method ($p = 4$), the extrapolated solution is

$$\hat{y}_{n+2} = y_{n+2} + \frac{y_{n+2} - w}{2^4 - 1} = y_{n+2} + \frac{y_{n+2} - w}{15}$$

This provides a fifth-order approximation, and the error estimate for y_{n+2} is $(y_{n+2} - w)/15$[1]. While this approach requires additional function evaluations compared to embedded formulas, it increases the order of the final approximation from 4 to 5.

2.4 Step Size Control Parameters

For all implementations, we used the following parameters in the step size control algorithm:

- Safety factor: $\text{fac} = 0.9$
- Minimum step size factor: $\text{fac}_{\text{min}} = 0.2$
- Maximum step size factor: $\text{fac}_{\text{max}} = 5.0$ (reduced to 1.0 after a rejected step)
- Error exponent: $1/(q+1)$ where q is the order of the error estimate

The tolerances Atol and Rtol were varied across test problems to analyze the performance characteristics of each method.

3 Numerical Experiments

Phase 1: Embedded Methods on Multiple Test Cases

To assess the behavior of embedded Runge–Kutta formulas in a general setting, we tested three different tableaux on multiple test problems. The selected methods were:

- **ERK4(3)**: A classical 4-stage method of order 4 with an embedded third-order estimator
- **BS3(2)**: The Bogacki–Shampine method of order 3 with a second-order embedded estimator
- **DP5(4)**: The Dormand–Prince method of order 5 with an embedded fourth-order estimator (the method used in MATLAB's `ode45`)

These methods were applied to three representative problems:

1. The two-dimensional Brusselator system exhibiting stiff-like behavior and oscillations
2. The scalar ODE $y' = 3 + \cos(t) - y$ with initial condition $y(0) = 1$
3. The scalar ODE $y' = y(2-y)(2+y)$ with initial condition $y(0) = 5$

Interactive MATLAB Application

An interactive MATLAB application was developed to facilitate systematic testing and parameter exploration. The application allows the user to select:

- The test equation (Brusselator, or either scalar ODE)
- The embedded tableau (ERK4(3), BS3(2), or DP5(4))
- Absolute tolerance (`Atol`) and relative tolerance (`Rtol`)
- Safety factor (`fac`) used in the step size controller

This tool enables exploration of the trade-off between accuracy and efficiency, and visualization of how different embedded pairs respond to changes in error tolerances and adaptive strategy aggressiveness. The application displays the solution trajectory, step size evolution (including accepted and rejected steps), and error behavior throughout the integration.

Figure 1: Comparison of adaptive step size control for ERK(3), ERK(4), and ERK(4) with Richardson extrapolation. Top: solution trajectories with accepted integration steps. Middle: accepted (connected) and rejected (isolated) step sizes. Bottom: estimated local error, exact local error, and global error.

Results from Multiple Test Cases

The three embedded methods were tested on all three ODEs with various tolerance settings. Key findings include:

- **DP5(4) superiority**: The Dormand-Prince method consistently outperformed lower-order pairs across all test problems, demonstrating excellent efficiency at moderate to high precision requirements
- **Problem-dependent behavior**: The scalar ODEs exhibited smoother step size evolution compared to the Brusselator, with fewer step rejections due to less chaotic dynamics
- **Tolerance sensitivity**: Tighter tolerances revealed the order advantage of DP5(4), while at loose tolerances BS3(2) remained competitive due to its lower per-step cost
- **Safety factor impact**: Larger safety factors (e.g., 0.9 vs. 0.8) reduced step rejections but increased total computational cost by being more conservative in step size increases

The interactive application proved valuable for understanding the interplay between method order, problem characteristics, and controller parameters in adaptive step size strategies.

Phase 2: Detailed Comparison on the Brusselator

For the second phase of analysis, we focused exclusively on the Brusselator system to conduct a detailed comparison between adaptive and non-adaptive strategies. The Brusselator is a well-known model from chemical kinetics exhibiting stiff behavior and oscillatory solutions[1]:

$$\begin{cases} y_1' = 1 + y_1^2 y_2 - 4y_1 \\ y_2' = 3y_1 - y_1^2 y_2 \end{cases}$$

with initial conditions $y_1(0) = 1.5$ and $y_2(0) = 3$, integrated over the interval $[0, 20]$ with tolerances $\text{Atol} = \text{Rtol} = 10^{-4}$.

3.1 Results and Analysis

The results demonstrate several key features of adaptive step size control:

- **Step size adaptation**: All three methods successfully adapted the step size to the local behavior of the solution, using smaller steps in regions of rapid variation (around $x \approx 11$) and larger steps in smooth regions

- **Step rejections**: Step rejections occurred primarily when the solution changed rapidly and the algorithm needed to decrease the step size. The ERK(4) method exhibited fewer rejections than ERK(3) due to its higher order

- **Error control**: The estimated local error remained close to the prescribed tolerance throughout the integration, validating the effectiveness of the step size control mechanism

- **Global error accumulation**: Despite tight local error control, global errors accumulated due to the chaotic sensitivity of the Brusselator system to initial conditions[1]

3.2 Performance Comparison

We compared the three implementations across a range of tolerances from 10^{-2} to 10^{-10} to generate work-precision diagrams:

Method	Function Evals	Global Error	Rejected Steps
ERK(3)	456	2.3×10^{-4}	42
ERK(4)	288	1.8×10^{-4}	28
ERK(4) + Richardson	384	4.2×10^{-5}	24

Table 4: Performance comparison for Atol = Rtol = 10^{-4}

Key observations:

- **ERK(4) efficiency**: The fourth-order method required significantly fewer function evaluations than ERK(3) for comparable accuracy, demonstrating the advantage of higher-order methods for moderate to high precision requirements

- **Richardson extrapolation**: The ERK(4) with Richardson extrapolation achieved the best accuracy for a given computational cost at high precision, though it required more function evaluations than the embedded formula approach due to the step doubling strategy

- **Low precision regime**: For loose tolerances (Rtol $> 10^{-3}$), ERK(3) was competitive with ERK(4) due to its lower cost per step

3.3 Adaptive vs. Non-Adaptive Method Comparison

We performed a detailed comparison on the Brusselator problem between the adaptive embedded solver and three fixed-order strategies:

- **ERK(3)**: A third-order explicit Runge-Kutta method with uniform step size

- **ERK(4)**: The classical fourth-order Runge-Kutta method with uniform step size

- **ERK(4) + Richardson**: A fourth-order ERK method enhanced by Richardson extrapolation to obtain fifth-order accuracy

The comparison focused on the global error at the final time versus the number of function evaluations, quantifying the efficiency gain provided by adaptive step size control over uniform step sizes.

For the Brusselator problem with tolerances Atol = Rtol = 10^{-4}, the adaptive embedded methods were approximately 2-3 times more efficient than their fixed step size counterparts, achieving comparable accuracy with significantly fewer function evaluations. The Richardson extrapolation approach demonstrated superior accuracy at high precision requirements, though at increased computational cost due to the step doubling strategy.

Key observations from the comparison:

- The DP5(4) method outperformed lower-order embedded pairs across all tolerance ranges
- Adaptive methods excel on problems with varying solution behavior, automatically concentrating computational effort in regions requiring higher resolution
- For loose tolerances, the overhead of step size control becomes comparable to the savings, reducing the efficiency advantage
- Richardson extrapolation remains valuable for verification purposes and when extremely high accuracy is required

The adaptive strategy's ability to reject steps and dynamically adjust based on local error estimates provides both reliability (maintaining prescribed tolerances) and efficiency (avoiding unnecessary fine resolution in smooth regions).

4 Conclusions

This work presented implementations of adaptive step size control for explicit Runge-Kutta methods of orders 3 and 4, including an enhancement using Richardson extrapolation. The key findings are:

- The automatic step size control mechanism based on embedded formulas and Richardson extrapolation effectively maintains prescribed error tolerances while minimizing computational cost
- Higher-order methods (ERK4) are generally more efficient than lower-order methods (ERK3) for moderate to high precision requirements
- Richardson extrapolation provides an elegant way to increase the order of accuracy and obtain error estimates, though at the cost of additional function evaluations
- The automatic initial step size selection algorithm successfully provides reasonable starting values without user intervention
- Adaptive methods significantly outperform fixed step size methods on problems with varying solution behavior

For practical computations, we recommend using embedded Runge-Kutta formulas of order 4 or higher (such as the Dormand-Prince 5(4) pair) as they provide an excellent balance between accuracy, efficiency, and robustness. Richardson extrapolation remains valuable for obtaining very high accuracy or for verifying the correctness of numerical solutions.

Future work could explore higher-order methods, dense output formulas for continuous solution approximation, and extensions to stiff differential equations using implicit or linearly implicit methods.

5 Exercises

The following exercises address both theoretical and practical aspects of explicit Runge–Kutta methods and adaptive step-size strategies[1].

A Show that Runge's method can be interpreted as the composition of two explicit Euler steps with step size $h/2$, followed by a Richardson extrapolation[1].

B Prove that no four-stage Runge–Kutta method of order four admits an embedded formula of order three[1].

C Show that the step-size control strategy is invariant under a rescaling of the independent variable. In particular, demonstrate that equivalent sequences of step sizes are generated when the strategy is applied to the following two initial value problems[1]:

$$y' = f(x, y), \quad y(0) = y_0, \quad y(x_{\text{end}}) = ?,$$

and

$$z' = \sigma f(\sigma t, z), \quad z(0) = y_0, \quad z\left(\frac{x_{\text{end}}}{\sigma}\right) = ?,$$

provided that the initial step sizes are chosen as h_0 and h_0/σ, respectively.

Solutions

Solution A

Consider Euler's method with full step size h for the initial value problem

$$y'(t) = f(t, y), \quad y(t_n) = y_n.$$

A single Euler step gives

$$y_{n+1} = y_n + hf(t_n, y_n).$$

If instead we use two half steps of size $h/2$, we obtain

$$y_{n+\frac{1}{2}} = y_n + \frac{h}{2} f(t_n, y_n),$$

$$y_{n+1} = y_{n+\frac{1}{2}} + \frac{h}{2} f\left(t_n + \frac{h}{2}, y_{n+\frac{1}{2}}\right),$$

so that

$$y_{n+1} = y_n + \frac{h}{2}\left[f(t_n, y_n) + f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2} f(t_n, y_n)\right)\right] \text{[1]}.$$

Let w denote the result of a single Euler step of size h,

$$w = y_n + hf(t_n, y_n),$$

which is a first-order method ($p = 1$)[1]. Applying Richardson extrapolation with

$$\hat{y}_{n+1} = y_{n+1} + \frac{y_{n+1} - w}{2^p - 1}$$

yields, for $p = 1$,

$$\hat{y}_{n+1} = 2y_{n+1} - w = y_n + hf\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}f(t_n, y_n)\right),$$

which is exactly Runge's two-stage second-order method[1].

Solution B

Let us consider a 4-stage Runge–Kutta method of order four and denote by b_j the weights of the primary method and by $\hat{b}_j$ those of a putative embedded third-order formula[1]. Define $d_j = b_j - \hat{b}_j$ for $j = 1, \ldots, 4$. The aim is to show that $d_j = 0$ for all j, which would imply $b_j = \hat{b}_j$ and thus contradict the assumption that the embedded formula has strictly lower order[1].

Introduce the matrices

$$U = \begin{pmatrix} b_2 & b_3 & b_4 \\ b_2c_2 & b_3c_3 & b_4c_4 \\ d_2 & d_3 & d_4 \end{pmatrix}, \quad V = \begin{pmatrix} c_2 & c_2^2 & \sum_j a_{2j}c_j - \frac{c_2^2}{2} \\ c_3 & c_3^2 & \sum_j a_{3j}c_j - \frac{c_3^2}{2} \\ c_4 & c_4^2 & \sum_j a_{4j}c_j - \frac{c_4^2}{2} \end{pmatrix},$$

where the entries of V are built from the Butcher coefficients (a_{ij}, c_i)[1]. Using the order conditions up to order four, one shows that

$$UV = \begin{pmatrix} \frac{1}{2} & \frac{1}{3} & 0 \\ \frac{1}{3} & \frac{1}{4} & 0 \\ 0 & 0 & 0 \end{pmatrix},$$

whose upper 2×2 block has nonzero determinant[1]. A linear algebra lemma then implies either $Ve_3 = 0$ or $U^\top e_3 = 0$, where $e_3 = (0, 0, 1)^\top$[1]. The first alternative contradicts the structure of the Butcher tableau (since $c_i \neq 0$ for $i \geq 2$), so necessarily $U^\top e_3 = 0$, which yields $d_2 = d_3 = d_4 = 0$[1].

Finally, using the consistency condition $\sum_{j=1}^4 b_j = \sum_{j=1}^4 \hat{b}_j = 1$, we obtain

$$\sum_{j=1}^4 d_j = 0 \quad \Rightarrow \quad d_1 = 0,$$

and therefore $b_j = \hat{b}_j$ for all j. This proves that a 4-stage order-4 Runge–Kutta method cannot admit an embedded order-3 formula[1].

Solution C

The adaptive step-size strategy determines the sequence of step sizes h_k from the normalized error estimate

$$\mathrm{err} = \sqrt{\frac{1}{n}\sum_{i=1}^n \left(\frac{y_{1,i} - \hat{y}_{1,i}}{\mathrm{sc}_i}\right)^2},$$

and updates the step length according to

$$h_{\text{new}} = h \left(\frac{1}{\text{err}} \right)^{1/(q+1)} [1],$$

possibly modified by safety factors and bounds[1].

Consider the two problems

$$y' = f(x, y), \quad y(0) = y_0,$$

and

$$z' = \sigma f(\sigma t, z), \quad z(0) = y_0,$$

with initial step sizes h_0 and h_0/σ, respectively[1]. For an embedded method of order p, the local error behaves as

$$\|y_k - \hat{y}_k\| \sim C h^{p+1} y^{(p+1)}(0), \quad \|z_k - \hat{z}_k\| \sim \tilde{C} \left(\frac{h}{\sigma} \right)^{p+1} z^{(p+1)}(0)[1].$$

Using the relation $z^{(p+1)}(0) = \sigma^{p+1} y^{(p+1)}(0)$, one obtains

$$\|z_k - \hat{z}_k\| \sim C h^{p+1} y^{(p+1)}(0),$$

so that the normalized errors for the two problems are asymptotically identical at each step[1]. Since the step-size update depends only on err, both problems generate the same sequence of step sizes, up to the initial rescaling of the independent variable. This shows that the step-size control strategy is invariant under time rescaling[1].

References

[1] E. Hairer, S. P. Nørsett, and G. Wanner, *Solving Ordinary Differential Equations I: Nonstiff Problems*, 2nd ed., Springer-Verlag, 1993.